Programación de Entornos Multimedia — Semana 5

Físicas 3D (Parte 1): Gravedad, Cuerpos Rígidos y Detección de Colisiones Básicas

Docente: Mg. Gabriel Guerrero Arellano

Módulo	Duración	Stack
Momento 2 — Físicas, Control y Carga de Assets	3 Horas (Teoría + Práctica Integrada)	Three.js + Cannon-es + Vite / ES Modules

Universidad Cooperativa de Colombia

Hilo Conductor — Conexión Momento 1 vs. Momento 2

Evolución del Curso

Momento 1 — Fundamentos y Gráficos

Semanas 1 a 4

- Renderizar la escena y posicionar geometrías
- Luces, sombras y materiales/texturas
- Animación mediante trigonometría o `requestAnimationFrame`

> Base visual sólida construida. Ahora la escena se ve bien, pero no reacciona.

🎯 Momento 2 — Sistemas Dinámicos (HOY)

Las animaciones por código manual son **limitadas e irrealistas** para interacción compleja.

Hoy le inyectamos **leyes de la física reales** a nuestro entorno multimedia para que los objetos:

- Reaccionen a la gravedad automáticamente
- Reboten con coeficientes de restitución configurables
- Colisionen entre sí sin código manual de trayectoria

El **Motor Físico:** Simulación del Mundo Real

CLASIFICACIÓN SEGÚN CAPACIDAD DE CÓMPUTO



Simulaciones en Tiempo Real (Videojuegos)

Prioriza velocidad, inmersión y respuesta inmediata sobre la precisión matemática perfecta.

VELOCIDAD

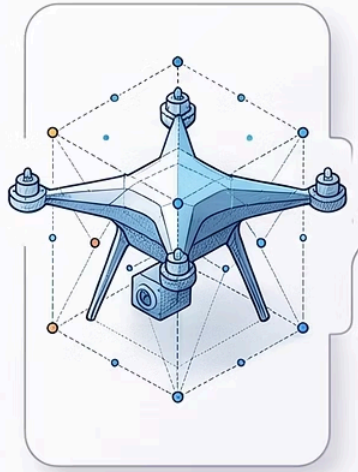
PRECISIÓN



Simulaciones de Alta Precisión (Científicas)

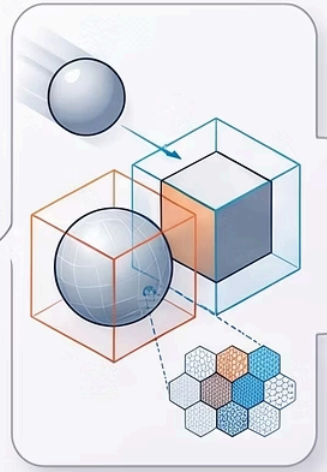
Requiere procesadores vectoriales y gran cómputo para modelar fenómenos con rigor exacto.

FUNCIONAMIENTO INTERNO Y DETECCIÓN



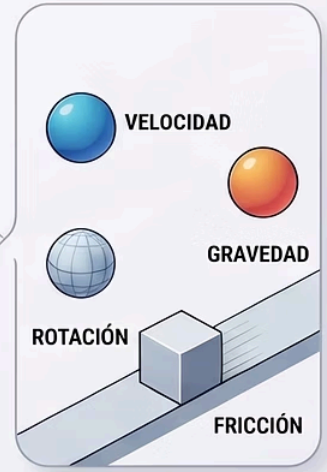
Mallas y Representación (Mesh)

Representación gráfica de objetos mediante la cantidad mínima de puntos y parámetros.



Detección de Colisiones

Evalúa intersecciones usando cajas imaginarias (Bounding Boxes) o teselado de volúmenes pequeños.



Leyes de Movimiento y Variables

Simula caídas y choques aplicando progresivamente gravedad, velocidad, rotación y fricción.

Gemini Notebook

Concepto 1 — ¿Por qué un Motor de Físicas?

Animación Cinemática vs. Simulación Física

Animación Manual (Cinemática)

Modificar `mesh.position.y -= 0.1` en cada frame.

Problema: No contempla aceleración, peso, inercia ni reacción ante obstáculos. El movimiento es lineal, artificial y no escala a escenas complejas.

Simulación Física (Dinámica)

Delegar el cálculo del movimiento a un motor matemático basado en $F = m \cdot a$.

Ventaja: El motor calcula trayectorias, rebotes, fricción y colisiones complejas de forma orgánica y automática.

Sin Motor Físico

- Posición actualizada manualmente
- Sin aceleración real
- Sin rebote ni fricción
- Colisiones imposibles de escalar

Con Motor Físico

- Ecuaciones de Newton integradas
- Gravedad, inercia y torque
- Rebote y fricción configurables
- Colisiones automáticas y precisas

Concepto 2 — La Arquitectura de Mundos Paralelos

El Secreto del Renderizado Físico 3D

Para lograr físicas en la web mantenemos **dos mundos independientes** corriendo simultáneamente en memoria:

MUNDO FÍSICO (Cannon.js)

Invisible en pantalla. Calcula masas, posiciones y vectores (x, y, z).

- Cuerpos rígidos (`CANNON.Body`)
- Formas de colisión (`Sphere`, `Box`, `Plane`)
- Gravedad, fricción y restitución
- Detección de contacto (Broadphase / Narrowphase)

MUNDO GRÁFICO (Three.js)

Visible para el usuario. Mallas (Meshes), Luces y Sombras.

- Geometrías y materiales visuales
- Iluminación y sombras dinámicas
- Cámara y renderer WebGL
- Efectos visuales y partículas

Misión del Desarrollador: En cada fotograma, copiar la posición y rotación del cuerpo físico al mesh gráfico — **Sincronización por Frame →**

Concepto 3 — El Bucle de Simulación (Fixed Time Step)

Cómo computa el tiempo el motor físico

1 El Problema de los FPS Variables

Las pantallas corren a tasas de refresco variables: 60 Hz, 120 Hz, 144 Hz. Si la física dependiera de los FPS de la pantalla, un juego iría al **doble de velocidad** en un monitor de 144 Hz.

2 El Paso de Tiempo Fijo (`World.step`)

Definimos un delta de tiempo constante, típicamente ***1/60*** de segundo (≈ 0.0166 s). Esto desacopla la física de la tasa de refresco del monitor.

3 Resultado: Consistencia Universal

El método `world.step(1 / 60, deltaTime, 3)` actualiza las ecuaciones de movimiento de forma **uniforme en cualquier dispositivo**.

Fórmula del Paso Fijo

t_físico = 1/60 segundos

≈ 0.0166 s por iteración

Parámetros de `world.step`

- `1/60` — Paso fijo de tiempo
- `deltaTime` — Tiempo real transcurrido
- `3` — Máx. sub-pasos por frame

Concepto 4 — Tipos de Cuerpos Rígidos (RigidBodies)

En un motor 3D, los objetos físicos se clasifican según su comportamiento frente a fuerzas:

Tipo de Cuerpo	¿Le afecta la gravedad?	¿Se mueve al colisionar?	Casos de Uso Ideal
Dynamic (Dinámico)	🟢 Sí	🟢 Sí	Cajas, pelotas, vehículos, proyectiles.
Static (Estático)	🔴 No	🔴 No	Pisos, paredes, rampas, terreno.
Kinematic (Cinemático)	🔴 No	🟢 Solo por código	Plataformas móviles, puertas automáticas.

Dynamic

`mass: 1` (o cualquier valor > 0). Responde a gravedad y colisiones.

Static

`mass: 0`. Inamovible. Ideal para el entorno del nivel.

Kinematic

Controlado por código. No responde a fuerzas externas.

Concepto 5 — Geometrías de Colisión (Bounding Shapes)

Rendimiento vs. Precisión

El cálculo de colisión entre polígonos complejos es extremadamente costoso para la CPU. Por ello, envolvemos los modelos visuales en formas físicas simplificadas:

Sphere

Mínimo costo computacional. Evalúa solo radio y centro. Ideal para pelotas, proyectiles y objetos aproximadamente redondos.

Box

Bajo costo. Caja orientada (Bounding Box). Perfecto para cajas, edificios, plataformas y objetos rectangulares.

Cylinder

Costo medio. Ideal para personajes, cápsulas y objetos cilíndricos como barriles o columnas.

Convex

Alto costo. Aproximación por malla poligonal. Usar solo cuando la precisión es crítica y el objeto es convexo.

Regla de oro: usa siempre la forma más simple que aproxime razonablemente el objeto visual. El rendimiento a 60 FPS depende de ello.

Concepto 6 — Propiedades de Materiales Físicos

CANNON.Material y CANNON.ContactMaterial

No se refieren a colores o texturas, sino a **comportamiento de superficie**:

Restitución (Restitution / Bounciness)

Factor de rebote al colisionar:

- **0.0** = Plastilina / sin rebote (energía absorbida totalmente)
- **0.5** = Pelota de tenis (rebote parcial)
- **1.0** = Pelota de goma / rebote perfecto (energía conservada)

Fricción (Friction)

Resistencia al deslizamiento sobre la superficie:

- **0.0** = Hielo superresbaladizo (sin resistencia)
- **0.3** = Superficie normal (madera, concreto)
- **1.0** = Lija / agarre total (máxima resistencia)

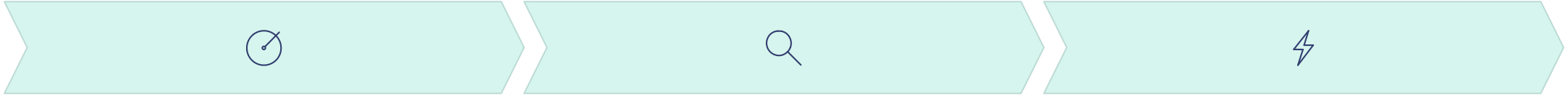
Ejemplo de Interacción entre Dos Superficies

```
// Ejemplo de interacción entre dos superficies
const contactMaterial = new CANNON.ContactMaterial(
  pisoMaterial, pelotaMaterial,
  { friction: 0.3, restitution: 0.7 }
);
```

El `ContactMaterial` define el comportamiento cuando **dos materiales específicos** entran en contacto. Se puede tener múltiples combinaciones en la misma escena.

Concepto 7 — Detección de Colisiones y Eventos

Fases del Algoritmo de Colisión



Broadphase

Fase Ancha. Filtra rápidamente qué objetos están lejos y descarta parejas imposibles de colisionar. Optimización crítica de rendimiento.

Narrowphase

Fase Estrecha. Evalúa el contacto exacto punto a punto de los objetos cercanos restantes. Alta precisión matemática.

Evento 'collide'

Escuchadores de Eventos. Disparar lógica multimedia: reproducir sonido 3D, emitir partículas o cambiar el color del objeto en colisión.

Ejemplo de Escuchador de Colisión

```
body.addEventListener('collide', (event) => {
 // Reproducir sonido, emitir partículas, cambiar color...
 const impactVelocity = event.contact.getImpactVelocityAlongNormal();
 if (impactVelocity > 1.5) playImpactSound();
});
```

Herramientas a Utilizar en el Taller

Stack Tecnológico



Runtime / Bundler

Node.js + Vite

Servidor ultrarrápido con Hot Module Replacement (HMR). Recarga instantánea al guardar cambios sin recargar la página completa.



Motor Físico

Cannon-es (cannon-es)

Fork moderno de Cannon.js con soporte completo para ES Modules. Simula gravedad, colisiones, fricción y restitución.



Motor Gráfico

Three.js (three)

Librería WebGL de alto nivel para renderizado 3D en el navegador. Gestiona escena, cámara, luces, materiales y geometrías.



Editor y Lenguaje

VS Code + JavaScript ES6+

Entorno de desarrollo con soporte para módulos ES, autocompletado y depuración integrada del navegador.

Laboratorio Práctico: Motor de Físicas 3D e Interacción

Integración de Renderizado Visual (Three.js) y Cálculo Físico (Cannon.es)

¿Qué vamos a construir?

Objetivo: Crear un entorno tridimensional interactivo con simulación de gravedad terrestre, colisiones elásticas, sombras dinámicas y generación (spawning) de cuerpos rígidos al hacer clic.

Stack Tecnológico:

- **Three.js:** Motor gráfico para renderizado de mallas 3D, cámara, luces y materiales en WebGL.
- **Cannon.es:** Motor de físicas 3D para cálculo de gravedad, vectores, colisiones y fricción.
- **Vite:** Servidor de desarrollo ágil para módulos ES de JavaScript.

Flujo de Trabajo Técnico

01

Paso 1

Configuración del entorno de desarrollo Node/Vite.

02

Paso 2

Inicialización del mundo gráfico y luces (Three.js).

03

Paso 3

Configuración del mundo físico y materiales de rebote (Cannon.es).

04

Paso 4

Sincronización marco a marco (frame-by-frame) en el bucle de animación.

05

Paso 5

Creación dinámica de objetos interactivos mediante eventos del navegador.

Laboratorio Paso 1 — Inicialización del Proyecto

Comandos de Configuración en Terminal de vscode (Vite):

```
npm init -y
npm install three cannon-es
npm install -D vite
npx vite
```

Estructura del Archivo index.html:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Laboratorio Físicas 3D - Semana 5</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div id="ui">Haz Clic en la pantalla para lanzar un objeto</div>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

💡 Explicación Didáctica para la Clase:

- **type="module":** Es obligatorio porque dentro de main.js usamos la sintaxis de importación de JavaScript moderno (import * as THREE...).
- **Vite:** Actúa como un servidor local que resuelve la carga de las librerías sin problemas de CORS ni rutas de archivos locales.

Laboratorio Paso 2 — Escena, Cámara y Luces

Código en main.js (Parte 1):

```
import * as THREE from 'three';
import * as CANNON from 'cannon-es';

// 1. Escena, Cámara y Renderizador
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight, 0.1, 100);
camera.position.set(0, 5, 10);
camera.lookAt(0, 0, 0);

const renderer = new THREE.WebGLRenderer({ antialias: true });
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.shadowMap.enabled = true; // Activa el cálculo de sombras
document.body.appendChild(renderer.domElement);

// 2. Iluminación
const ambientLight = new THREE.AmbientLight(0xffffff, 0.5);
scene.add(ambientLight);

const dirLight = new THREE.DirectionalLight(0xffffff, 1);
dirLight.position.set(5, 10, 5);
dirLight.castShadow = true; // Permite proyectar sombras sobre el suelo
scene.add(dirLight);
```

💡 Explicación Didáctica para la Clase:

- **PerspectiveCamera:** Imita el ojo humano (campo de visión de 75° y perspectiva de profundidad).
- **camera.lookAt(0,0,0):** Apunta el lente de la cámara exactamente al centro del suelo tridimensional.
- **shadowMap.enabled = true:** Sin este parámetro, las sombras no se calculan y los objetos parecen "flotar" sin sentido de profundidad.

Laboratorio Paso 3 — Gravedad y Materiales de Contacto

Código en main.js (Parte 2):

```
// 1. Creación del Mundo Físico
const world = new CANNON.World();
world.gravity.set(0, -9.82, 0); // Simulación de la gravedad en m/s² (Eje Y)

// 2. Definición de Propiedades Físicas (Fricción y Elasticidad)
const defaultMaterial = new CANNON.Material('default');
const contactMaterial = new CANNON.ContactMaterial(
  defaultMaterial,
  defaultMaterial,
  {
    friction: 0.4,    // Resistencia al deslizamiento
    restitution: 0.7  // Coeficiente de rebote (0 = Plastilina, 1 = Pelota de goma)
  }
);
world.addContactMaterial(contactMaterial);
world.defaultContactMaterial = contactMaterial;
```

💡 Explicación Didáctica para la Clase:

- **world.gravity.set(0, -9.82, 0):** Define la fuerza hacia abajo en el eje vertical Y.
- **restitution: 0.7:** Controla la energía que conserva el objeto al colisionar. Un valor de 0.7 genera un rebote elástico realista sin ser infinito.

Laboratorio Paso 4 — Construcción del Plano Base

Código en main.js (Parte 3):

```
// --- SUELO FÍSICO (CANNON.ES) ---
const groundShape = new CANNON.Plane();
const groundBody = new CANNON.Body({ mass: 0, material: defaultMaterial }); // Masa 0 = Objeto inamovible
groundBody.addShape(groundShape);
groundBody.quaternion.setFromEuler(-Math.PI / 2, 0, 0); // Rotar -90 grados en X
world.addBody(groundBody);

// --- SUELO GRÁFICO (THREE.JS) ---
const groundMesh = new THREE.Mesh(
    new THREE.PlaneGeometry(20, 20),
    new THREE.MeshStandardMaterial({ color: 0x222222 })
);
groundMesh.rotation.x = -Math.PI / 2; // Orientar paralelo al suelo físico
groundMesh.receiveShadow = true;
scene.add(groundMesh);
```

💡 Explicación Didáctica para la Clase:

- **¿Por qué dos suelos?:** Three.js dibuja lo que se ve (groundMesh), mientras que Cannon.es calcula dónde rebotan las cosas (groundBody).
- **mass: 0:** Le indica al motor matemático que este cuerpo no cae ni se mueve cuando un objeto pesado lo golpea.
- **quaternion.setFromEuler(-Math.PI / 2, 0, 0):** Por defecto los planos en 3D están verticales (como paredes). Los rotamos 90° para dejarlos horizontales.

Laboratorio Paso 5 — Sincronización de Coordenadas

Código en main.js (Parte 4):

```
const objectsToUpdate = []; // Colección de pares { mesh, body }
const clock = new THREE.Clock();
let oldElapsedTime = 0;

function animate() {
    requestAnimationFrame(animate);

    const elapsedTime = clock.getElapsedTime();
    const deltaTime = elapsedTime - oldElapsedTime;
    oldElapsedTime = elapsedTime;

    // 1. Avanzar la simulación física (paso de tiempo fijo a 60 FPS)
    world.step(1 / 60, deltaTime, 3);

    // 2. Copiar la posición y rotación matemática de Cannon a las mallas de Three
    for (const object of objectsToUpdate) {
        object.mesh.position.copy(object.body.position);
        object.mesh.quaternion.copy(object.body.quaternion);
    }

    renderer.render(scene, camera);
}

animate();
```

💡 Explicación Didáctica para la Clase:

- **world.step(1 / 60, ...):** Cannon.es calcula dónde debe estar cada objeto en el siguiente milisegundo.
- **object.mesh.position.copy(...):** El "puente" que une la física con los gráficos. Mueve el modelo 3D visible exactamente a donde Cannon dice que cayó el objeto invisible.

Laboratorio Paso 6 — Creación Dinámica de Esferas

Código en main.js (Parte 5):

```
function createSphere(radius, position) {
    // 1. Malla Gráfica
    const mesh = new THREE.Mesh(
        new THREE.SphereGeometry(radius, 32, 32),
        new THREE.MeshStandardMaterial({
            color: Math.random() * 0xffffff, // Color aleatorio
            roughness: 0.2
        })
    );
    mesh.castShadow = true;
    mesh.position.copy(position);
    scene.add(mesh);

    // 2. Cuerpo Físico
    const shape = new CANNON.Sphere(radius);
    const body = new CANNON.Body({
        mass: 1, // Objeto dinámico con peso
        shape: shape,
        position: new CANNON.Vec3(position.x, position.y, position.z)
    });
    world.addBody(body);

    // 3. Vincular para actualización continua
    objectsToUpdate.push({ mesh, body });
}
```

💡 Explicación Didáctica para la Clase:

- **Math.random() * 0xffffff:** Genera un color HEX distinto para cada esfera que el usuario lanza.
- **objectsToUpdate.push({ mesh, body }):** Registra la pareja en la lista para que la función animate() sepa que debe sincronizar este nuevo objeto en cada fotograma.

Laboratorio Paso 7 — Disparador Clic y Ajuste Responsivo

Código en main.js (Parte 6):

```
// Evento: Lanzar esfera al hacer clic
window.addEventListener('click', () => {
  createSphere(
    0.4 + Math.random() * 0.3, // Radio variable entre 0.4 y 0.7
    {
      x: (Math.random() - 0.5) * 3, // Dispersión horizontal X
      y: 8,                         // Caída desde altura Y = 8
      z: (Math.random() - 0.5) * 3  // Dispersión profundidad Z
    }
  );
});

// Evento: Reajuste automático de pantalla
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

ℹ️ 💡 Explicación Didáctica para la Clase:

- **(Math.random() - 0.5) * 3:** Hace que las esferas no caigan exactamente en la misma coordenada, permitiendo que colisionen entre ellas al llegar al suelo.

Verificación del Laboratorio en el Navegador

Matriz de Pruebas y Evidencias

Prueba Realizada	Acción en Pantalla	Resultado Esperado en Simulación
Prueba de Renderizado	Cargar `http://localhost:5173`	Plano oscuro visible, sombras activas y texto flotante.
Prueba de Físicas	Clic continuo en pantalla	Generación de esferas de colores que caen, rebotan y se empujan entre sí.
Auditoría de Consola (F12)	Escribir `objectsToUpdate`	Arreglo de objetos activos incrementándose en tiempo real por cada clic.

Conclusiones y Puente hacia la Semana 6

Puntos Clave Consolidados

✅ Arquitectura de Mundos Paralelos

La simulación física requiere **desacoplar el mundo gráfico (Three)** del mundo matemático (Cannon). Ambos corren en paralelo y se sincronizan por frame copiando `position` y `quaternion`.

✅ Materiales de Contacto

Los **ContactMaterial** controlan el realismo visual mediante el rebote (`restitution`) y la fricción (`friction`). Son la clave para que la simulación se sienta orgánica y creíble.

✅ Sincronización en requestAnimationFrame

La sincronización se realiza copiando `position` y `quaternion` en **cada fotograma** dentro del bucle `requestAnimationFrame`, usando un paso de tiempo fijo de 1/60 s.

🚀 Próxima Clase — Semana 6: Físicas Avanzadas y Control

Aplicaremos **impulsos y fuerzas direccionales** para construir un personaje 3D o vehículo totalmente controlable mediante el teclado.

- Control WASD para movimiento en el plano XZ
- Barra Espaciadora para salto con impulso vertical
- Detección de estado en suelo (grounded check)
- Cámara en tercera persona siguiendo al personaje

Resumen del Stack Dominado Hoy

Three.js

Escena, cámara, luces, sombras, geometrías y renderer WebGL.

Cannon-es

World, Body, Shape, Material, ContactMaterial y eventos de colisión.

Vite

Bundler moderno con HMR para desarrollo ágil con ES Modules.